Session 13&14

Part01

Batch Normalization

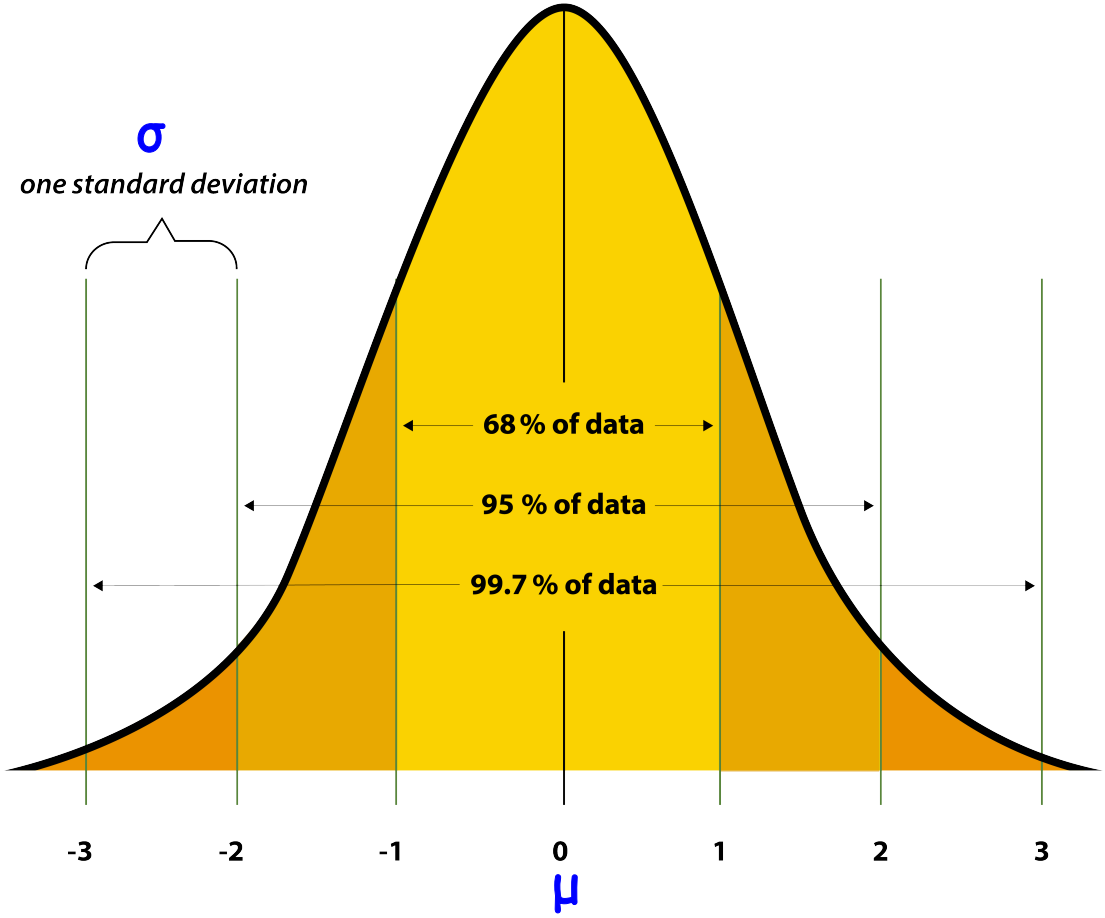
Deep Learning | Hobot | Zahra Amini

Hobot Academy

Telegram: @zahraamini_ai & Instagram:@zahraamini_ai & LinkedIn: @zahraamini-ai

https://aminizahra.github.io

Normal Distributions



$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- μ is the mean of the distribution
- σ^2 is the variance of the distribution
- σ is the standard deviation of the distribution

$$\mu = \frac{1}{m} \sum_{i=1}^{m} x_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu)^2$$

:Batch Normalization

بچ نرمالیزیشن داده‌های ورودی به هر لایه از شبکه عصبی را نرمالیزه می‌کند. این داده‌ها می‌توانند شامل

ویژگی‌های ورودی در لایه‌های ابتدایی و یا خروجی‌های میانی لایه‌های شبکه باشند

Batch Normalization چه کاری انجام می دهد؟

بچ نرمالیزیشن ویژگی‌های هر لایه را نرمالیزه می‌کند تا میانگین آن‌ها صفر و واریانس آن‌ها یک شود

فرآیند Batch Normalization:

1. محاسبه میانگین و واریانس بچ ---> برای هر بچ از داده‌های ورودی، میانگین(μ) و واریانس(σ^2) محاسبه می‌شود

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i-\mu)^2$$

2. نرمالیزه کردن داده‌ها ---> داده‌های ورودی بچ با استفاده از میانگین و واریانس محاسبه شده، نرمالیزه می‌شوند

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2+\varepsilon}}$$

- x_i is the i-th input
- where ϵ is a small constant added to avoid division by zero

3 مقیاس‌بندی و انتقال ---> داده‌های نرمالیزه شده سپس با استفاده از پارامترهای قابل آموزش β و γ مقیاس‌بندی و انتقال می‌یابند

$$y_i = \gamma \hat{x}_i + \beta$$

بچ نرمالیزیشن در لایه‌های مختلف شبکه عصبی

لایه‌های ورودی ---> ویژگی‌های ورودی نرمالیزه می‌شوند تا شبکه با داده‌هایی با مقیاس یکنواخت مواجه شود

لایه‌های میانی ---> خروجی لایه‌های میانی نرمالیزه می‌شود تا تغییرات شدید در مقادیر خروجی هر لایه کاهش یابد و شبکه به صورت یکنواخت‌تری آموزش ببیند

لایه‌های خروجی ---> در بعضی از موارد، نرمالیزه کردن خروجی نهایی نیز می‌تواند کمک‌کننده باشد، اما این بستگی به نوع مسئله و معماری شبکه دارد

✔ بچ نرمالیزیشن ویژگی‌های هر لایه از شبکه عصبی را نرمالیزه می‌کند تا فرآیند آموزش پایداری بیشتری داشته باشد و شبکه بتواند سریع‌تر و با دقت بیشتری بهینه شود

Batch Normalization

$$X = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 4 & 5 & 6 & 1 & 2 \\ 7 & 8 & 9 & 2 & 3 \\ 0 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * K = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} \dashrightarrow \begin{bmatrix} 28.5 & 15.3 & 12.1 \\ 16.5 & 15.5 & 12.7 \\ 10.6 & 12.7 & 13.1 \end{bmatrix}$$

$$(1\times0.1+2\times0.2+3\times0.3+4\times0.4+5\times0.5+6\times0.6+7\times0.7+8\times0.8+9\times0.9)$$

$$= 0.1 + 0.4 + 0.9 + 1.6 + 2.5 + 3.6 + 4.9 + 6.4 + 8.1 = 28.5$$

Calculate Mean (μ) and Variance (σ^2)

$$\mu = \frac{1}{9}(28.5 + 15.3 + 12.1 + 16.5 + 15.5 + 12.7 + 10.6 + 12.7 + 13.1) \approx 15.22$$

$$\sigma^2 = \frac{1}{9}((28.5-\mu)^2 + (15.3-\mu)^2 + (12.1-\mu)^2 + (16.5-\mu)^2 + (15.5-\mu)^2 + (12.7-\mu)^2 + (10.6-\mu)^2 + (12.7-\mu)^2 + (13.1-\mu)^2)$$

$$= \frac{1}{9}((28.5-15.22)^2 + (15.3-15.22)^2 + (12.1-15.22)^2 + (16.5-15.22)^2 + (15.5-15.22)^2 + (12.7-15.22)^2 + (10.6-15.22)^2 + (12.7-15.22)^2 + (13.1-15.22)^2) \approx 25.24$$

Normalize the Outputs ($\hat{z}$) Using a small ϵ=0.00001

For z_1 = 28.5

$$\hat{z}_1 = \frac{28.5 - \mu}{\sqrt{\sigma^2 + \epsilon}} = \frac{28.5 - 15.22}{\sqrt{25.24 + 0.00001}} \approx 2.65$$

Scale and Shift (y), Assume γ=1 and β = 0

For $\hat{z}_1$ = 2.65

$$y = \gamma\, \hat{z}_1 + \beta = 1 \times 2.65 + 0 = 2.65$$

$$\begin{bmatrix} 2.65 & 0.02 & -0.62 \\ 0.26 & 0.06 & -0.5 \\ -0.92 & -0.5 & -0.42 \end{bmatrix} \xrightarrow{\text{ReLU}} \begin{bmatrix} 2.65 & 0.02 & 0.0 \\ 0.26 & 0.06 & 0.0 \\ 0.0 & 0.0 & 0.0 \end{bmatrix}$$

مقادیر γ، β چه طور بدست می آیند؟

1. وقتی یک لایه بچ‌نرمالیزیشن به مدل در کراس اضافه می‌شود، γ، β مقداردهی اولیه می‌شوند

به صورت پیش‌فرض 👇

γ به یک بردار از یک‌ها مقداردهی اولیه می‌شود

β به یک بردار از صفرها مقداردهی اولیه می‌شود

2. با استفاده از یک الگوریتم بهینه‌سازی مانند SGD، Adam به‌روزرسانی می‌شوند تا هزینه به حداقل برسد

$$\gamma \leftarrow \gamma - \alpha \frac{\partial L}{\partial \gamma}$$

$$\beta \leftarrow \beta - \alpha \frac{\partial L}{\partial \beta}$$

Batch Normalization in Keras

```
# Define the model
model = Sequential()

# Add a single convolutional layer
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(32, 32, 3)))  # Assuming RGB
model.add(BatchNormalization())
model.add(Activation('relu'))

# Add a max-pooling layer
model.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten the output
model.add(Flatten())

# Add a fully connected layer
model.add(Dense(128))
model.add(BatchNormalization())
model.add(Activation('relu'))

# Add the output layer
model.add(Dense(10, activation='softmax'))  # Assuming 10 classes for output

# Compile the model
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
```